

CSE / EEE / ETE 499A (Section 1)

Project Management and Financial Viability Report (CO5)

Project Title:

ShopMate — AI A Catalog-Grounded, Omnichannel AI Sales & Support Agent for Facebook, Instagram, WhatsApp, and Website Commerce

Submitted To

Dr. Shazzad Hosain (SZZ)

Date: 02/08/2026



Semester: Summer 2026

Group No: G-1

Group Members

ID	Name
2311966642	Safat Uddin
2311790642	Farhan Karim
2311443042	Safwan Ismaun Amin
2311195042	Mahir Shahriyar Adib

1. Introduction and Project Objectives

1.1 Background and Problem Statement

Small and medium e-commerce merchants in Bangladesh particularly the large “f-commerce” segment that sells exclusively through a Facebook Page, Instagram profile, or WhatsApp number depend entirely on manual, human-answered chat threads across fragmented channels. This produces three compounding problems: fragmented inboxes and lost sales when a reply is slower than a competitor's, a disproportionate amount of staff time spent on repetitive Tier-1 questions (price, stock, delivery), and a lack of trustworthy AI tooling, since most existing chatbot builders either free-generate answers (risking a wrong price or an unkeepable promise) or do not support Bangla/Banglish well enough for this specific market.

1.2 Proposed Solution

ShopMate AI is a multi-tenant, catalog-grounded, omnichannel AI sales-and-support agent that unifies Facebook Messenger, Instagram DM/comments, WhatsApp Business, and a website chat widget into a single merchant inbox; generates replies grounded in the merchant's own product catalog via Retrieval-Augmented Generation (RAG) rather than free-generating potentially inaccurate answers; detects purchase intent and complaints to build in-chat carts, create orders, and route escalations to a human; and meters AI usage per tenant with a merchant-facing analytics dashboard.

1.3 Project Objectives

- Develop a functional, end-to-end multi-channel AI sales agent prototype spanning at least Facebook Messenger, WhatsApp, and a website widget.
- Automate and measurably improve response speed and accuracy for repetitive, high-volume Tier-1 customer questions.
- Ensure the system is usable and trustworthy for non-technical SME merchants, with human-in-the-loop review of every AI-drafted reply.
- Ensure reliability through catalog-grounded generation that eliminates hallucinated prices, stock levels, or delivery promises.
- Complete design, development, testing, and documentation of the project within the fixed seven-month CSE499A/CSE499B window and the cost envelope defined in Section 11.

2. Scope of the Project

2.1 In-Scope Features

- Channel adapters for the website chat widget, Facebook Messenger, WhatsApp Business (Cloud API), and Instagram DM/comments.
- Signature-verified, deduplicated webhook ingestion and a channel-agnostic unified message contract.
- Catalog Service with product/variant sync (Shopify/WooCommerce) and a RAG retrieval pipeline grounding every AI reply in real catalog data.
- Language detection and native Bangla / English / Banglish reply generation.
- Complaint/intent classification with automatic routing of complaints to a human-follow-up queue.
- In-chat cart building and one-click order creation, visible on a merchant order-tracking view.
- Per-tenant AI-message usage metering and a merchant analytics dashboard (automation rate, response time, order uplift).

- Configurable AI persona (tone of voice, response style) exposed to the merchant without requiring code changes.

2.2 Out-of-Scope Features

- Native mobile applications for merchants or customers (the dashboard is web-only for this project's duration).
- Image-based product recognition from a customer-sent photo, suggestive AI/upsell recommendations, and TikTok Shop integration (identified as stretch goals in the CO1 proposal, Section 3.5, but deferred beyond the seven-month window).
- Physical logistics/shipping/courier integration and fulfillment tracking beyond order-status fields already present in the merchant's e-commerce backend.
- Enterprise features such as single sign-on (SSO), multi-organization hierarchies, or custom on-premise deployment; the system targets SME-scale, cloud-hosted multi-tenancy only.
- Support for languages beyond Bangla, English, and Banglish.
- Production-grade payment processing beyond generating a secure checkout link through a third-party payment gateway (no in-house card handling).

3. Stakeholders and Team Structure

3.1 Stakeholders

- **End users (customers)** the shoppers messaging a merchant's Facebook/Instagram/WhatsApp/website, who receive AI-generated replies and place orders through the chat.
- **Merchants (primary client persona)** Bangladeshi f-commerce SME owners who configure, monitor, and act on the system via the dashboard; their trust and adoption define project success (CO1 proposal, Section 3.5).
- **Supervisor** Dr. Shazzad Hosain (SZZ), who evaluates the project against the CO1–CO5 course outcomes and provides technical/academic guidance.
- **Development team** the four-member student group (Section 3.2), responsible for design, implementation, testing, and documentation.
- **Academic institution** North South University, as the degree-granting body whose capstone requirements this report and its companion CO1/CO2/CO4 reports satisfy.
- **External platform providers (indirect stakeholders)** Meta (Messenger/Instagram/WhatsApp), the LLM API provider, and the cloud/hosting provider, whose policies and pricing directly constrain the system (CO4, Sections 1.2 and 4).

3.2 Team Roles

Team Member	Role	Responsibility
[Name 1]	Backend / Infrastructure Lead	Ingestion gateway, message bus, core services, deployment (Docker/CI), database schema.
[Name 2]	AI / ML Lead	RAG pipeline, LLM Gateway, language detection, complaint/intent classifiers, guardrails.

[Name 3]	Commerce & Channel Lead	Channel adapters (Messenger/WhatsApp/Instagram/widget), Catalog and Order & Cart services.
[Name 4]	Frontend / QA Lead	Merchant dashboard, test-catalog seeding, integration testing, documentation.
Rotating (all members)	Project Manager (rotating)	Sprint planning, task tracking, supervisor reporting rotated so no single member is responsible for both building and grading-critical coordination.

4. Project Management Plan

- **Development methodology** Agile/Scrum with two-week sprints, chosen because the seven-month, two-course timeline (CO1, Section 4.4) naturally maps onto roughly 14–15 sprints and lets the team re-prioritize the “nice-to-have” backlog (CO1, Section 3.5) without disrupting the fixed CSE499A/CSE499B milestone dates.
- **Task allocation** work is tracked as issues on a GitHub Projects Kanban board (Backlog → In Progress → In Review → Done), pre-assigned by functional role (Section 3.2) for the critical-path items identified in the CO1 proposal (Section 4.3), with parallelizable items picked up flexibly once the shared message contract and catalog schema are frozen.
- **Progress monitoring** a weekly sprint check-in reviews the Kanban board and a simple burndown against the timeline in Section 6; any task more than one sprint behind triggers a scope conversation (defer to the nice-to-have backlog) rather than silently slipping the CSE499A/B milestone dates.
- **Meeting and reporting process** weekly internal team stand-ups, a biweekly supervisor progress meeting, and end-of-sprint demo reviews; all CO1–CO5 report drafts are reviewed by the full team before submission, with authorship rotated per the CO1 proposal's people-management plan (Section 4.5) so no single member owns both building and writing.

5. Work Breakdown Structure (WBS)

The project is broken into six major phases, each further decomposed into the concrete modules that map directly onto the system architecture defined in the CO2 design report.

1. **Requirement Analysis** stakeholder and market analysis (CO1, Sections 1–3); feature scoping into must-have vs. nice-to-have (CO1, Section 3.5).
2. **System Design** architecture and sub-component design, database/ER design, and UX/UI design (CO2 report, Sections 3, 4, and 2 respectively).
3. **Development** (a) Ingestion Gateway and unified message contract; (b) Catalog Service and RAG retrieval pipeline; (c) channel adapters for Facebook, WhatsApp, Instagram, and the website widget; (d) Order & Cart service and complaint routing; (e) metering, billing, and the merchant dashboard.
4. **Testing** unit and integration testing per service; RAG reply-accuracy and complaint-classifier recall evaluation against the seeded test catalog (CO1, Section 3.5 success metrics); load and per-tenant rate-limit testing.
5. **Documentation** CO1–CO5 course-outcome reports and internal technical documentation (API contracts, schema dictionary, runbook).
6. **Deployment** staging environment deployment for the CSE499A demo milestone and the CSE499B final defense demo.

6. Project Timeline and Milestones

Task	Start Date	End Date
Requirement Analysis	01 Sep 2026	30 Sep 2026
Design (Architecture, DB, UX)	15 Sep 2026	20 Oct 2026
Development Phase 1 (CSE499A: widget + Messenger MVP)	01 Oct 2026	05 Dec 2026
Testing CSE499A Integration & Demo	01 Dec 2026	20 Dec 2026
Development Phase 2 (CSE499B: WhatsApp, Instagram, Orders, Dashboard)	05 Jan 2027	20 Mar 2027
Testing System Integration & Performance	15 Mar 2027	10 Apr 2027
Final Submission (Documentation & Defense)	20 Mar 2027	25 Apr 2027

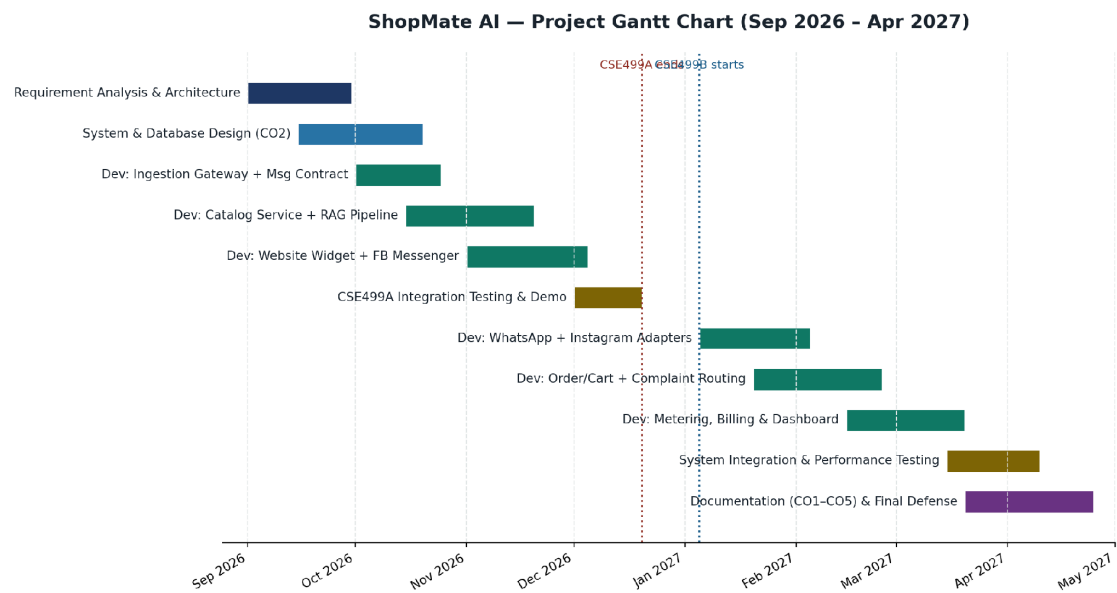


Figure 6.1 Project Gantt Chart (also see Appendix, Section 16)

7. Resource Planning

Hardware Resources

- Personal laptops/desktops for all four team members (development, testing, and demo delivery).
- Personal smartphones (Android/iOS) with active WhatsApp/Messenger/Instagram accounts, used as real devices for end-to-end channel testing.
- No dedicated physical server is required; all hosting runs on managed cloud infrastructure (below).

Software Resources

- **IDE / Dev tools** Visual Studio Code, Git/GitHub, Docker Desktop, Postman/Insomnia for API testing.
- **Database** PostgreSQL with the pgvector extension (transactional + embeddings), Redis (cache/session/rate-limiting).

- **Frameworks / runtime** Node.js/Next.js and Python services, per the polyglot service boundaries defined in the CO2 architecture.
- **AI / LLM** LLM Gateway API access (tiered cheap/frontier models per CO2, Section 6.6).
- **Cloud / hosting** AWS Free Tier and AWS Educate/student credits for compute, managed Postgres/Redis, and object storage (CO1, Section 4.6).
- **Third-party sandboxes** Meta for Developers test app and WhatsApp Cloud API sandbox numbers (free for development/testing).
- **Design / project management** Figma for UI mockups, draw.io/Lucidchart for diagrams, GitHub Projects for Kanban tracking.

8. Risk Management Plan

Risk	Impact	Mitigation Strategy
Development delay	High	Weekly sprint check-ins against the Section 6 timeline; any task slipping more than one sprint is descoped to the nice-to-have backlog (CO1, Section 3.5) rather than delaying a milestone.
Platform policy risk (Meta/WhatsApp verification, API changes)	High	Early sandbox setup in Month 1 (Section 6); Integrations Hub UI proactively surfaces token-expiry/re-authentication needs (CO2, Section 2.1).
LLM hallucination / trust failure	High	Hard grounding rule enforced by the Guardrail sub-component (CO2, Section 3.2), with a safe fallback template on failed grounding checks.
Resource / cost overrun (LLM API usage)	Medium	Tiered model routing (CO2, Section 6.6) and per-tenant rate limiting keep average cost bounded; monthly cost tracked against the Section 11.1 budget.
Team coordination issues	Medium	Weekly team stand-ups, rotating project-manager role (Section 3.2), and a shared Kanban board for visibility.
Data privacy incident	Medium	Only seeded synthetic merchant/customer data used in development and demos, never live customer PII (CO1, Section 3.6; CO4, Section 3.3).
Scope creep	Medium	Explicit in-scope/out-of-scope definition (Section 2) reviewed at each sprint planning session before new work is accepted.

9. Quality Assurance Plan

- **Testing strategy** unit tests per service (Catalog, Conversation, Order & Cart, Metering), integration tests against sandboxed channel webhooks, and end-to-end tests covering the full sequence-diagram flow in the CO2 report (Section 5.2).
- **Performance evaluation** RAG reply accuracy measured against the seeded test catalog (target $\geq 90\%$ on price/stock questions, per the CO1 success factors, Section 3.5), complaint-classifier precision/recall on a labelled test set, and average end-to-end reply latency tracked against a defined budget.
- **Reliability verification** idempotency tests (duplicate webhook delivery must not create duplicate messages/orders), rate-limit and load testing per tenant, and grounding-guardrail regression tests to confirm ungrounded claims are consistently rejected before every release.

10. Communication Plan

Communication Type	Frequency
Team Stand-up (async or short sync)	Weekly
Supervisor Progress Meeting	Biweekly
Sprint Demo / Review	End of each 2-week sprint
Async team channel (Slack/Discord)	Daily, as needed
Ad-hoc blocker/incident escalation	As needed

11. Financial Viability Analysis

11.1 Estimated Cost

Item	Estimated Cost (7 months)
Software / Tools (LLM API usage, tiered)	~\$140 – \$420 ($\approx$ \$20–\$60/month)
Hardware	\$0 (personal laptops/phones already owned by the team)
Hosting / Cloud (AWS Free Tier + student credits)	~\$0 – \$350 ($\approx$ \$0–\$50/month)
Miscellaneous (domain, misc. testing numbers)	~\$25 one-time
Estimated Total	~\$165 – \$795

11.2 Cost-Benefit Analysis

At an estimated total prototype cost of under \$800 across the full seven-month capstone (Section 11.1), the project is inexpensive to build relative to the value it is designed to demonstrate: the CO1 proposal's success metrics (Section 3.5) target measurable reductions in average response time and a measurable order-uplift percentage, both of which translate directly into merchant revenue once even a handful of pilot merchants adopt the system. For context, if a single pilot merchant's order uplift (target +28%, matching the illustrative figure shown on the Analytics Dashboard in the CO2 report, Figure 2.1) is realized on even a modest existing monthly order volume, the resulting incremental revenue for that one merchant would exceed the system's entire monthly operating cost many times over the benefit scales per merchant while the marginal cost per additional merchant (mostly metered LLM calls) grows much more slowly due to the tiered-model-routing cost control (CO2, Section 6.6).

11.3 Financial Feasibility

For the capstone phase, the project is clearly financially practical: nearly all infrastructure is covered by free-tier cloud credits and free developer sandboxes, and the only recurring cost metered LLM usage is small and directly controllable via the tiered routing strategy. Beyond the capstone, financial feasibility for a real commercial launch would depend on converting this prototype-scale cost structure into a per-tenant subscription price (e.g., in the \$15–\$30/month range typical of comparable SME chatbot tools reviewed in the CO1 literature review, Section 2.3) that comfortably exceeds the fully-loaded per-tenant LLM and infrastructure cost; at the tiered-routing cost estimates in Section 11.1, even a small number of paying pilot merchants would be sufficient to cover ongoing operating cost, making a post-graduation transition to a sustainable, self-funding product a realistic next step rather than a purely academic exercise.

16. Appendices

Appendix A: Gantt Chart

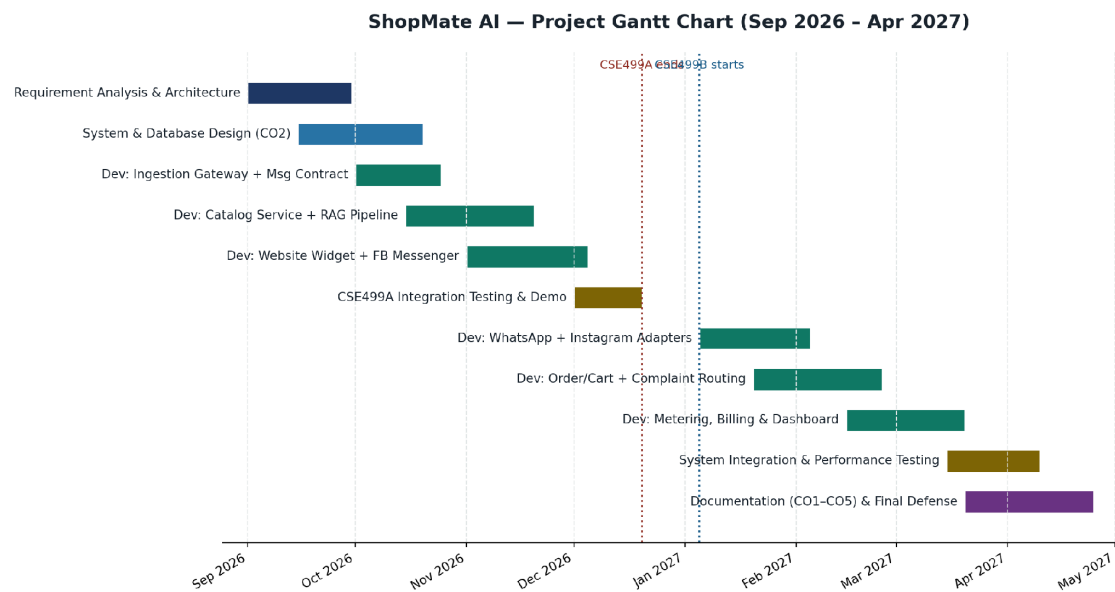


Figure A.1 Full Project Gantt Chart (Sep 2026 – Apr 2027)

Appendix B: Budget Details

See Section 11.1 for the full itemized cost table. The dominant variable cost is metered LLM API usage; hardware and most software tooling are covered by resources the team already owns or free developer/student tiers, as detailed in Section 7.

Appendix C: Design Diagrams

The system architecture diagram, use case diagram, entity-relationship diagram, class diagram, and sequence diagram referenced throughout this report are provided in full in the CO2 Design Report (Sections 1, 3, 4, and 5) and are not duplicated here.